

Glass Group Control Center

SOFTWARE DE CONTROL DE NAVES TRIPULADAS

Parte I

Este documento no fue escrito con IA

2026

INTRODUCCIÓN

En un futuro en el que la exploración espacial se ha extendido a regiones cada vez más alejadas, las naves interestelares deben desarrollar misiones durante períodos prolongados y en entornos que pueden cambiar de manera imprevisible. Las grandes distancias hacen que las comunicaciones con las bases de operaciones no siempre sean inmediatas. Por esta razón, cada nave cuenta con una tripulación comandada por un capitán y con un software embarcado capaz de asistirle durante la preparación, ejecución y evaluación de sus misiones.

El presente proyecto se centra en el desarrollo del Centro de Control espacial (G.G.C.C), quien se encarga de comunicarse directamente con los distintos Asistentes de Comando que operan sobre la nave, accediendo así al control de Recursos, asignación de distintas Misiones y sus respectivos registros históricos de eventos que sufren los distintos componentes de la nave, como su Motor Warp.

Dicho software se desarrolla mediante el lenguaje Java. implementando patrones y métodos necesarios para la correcta ejecución del software.

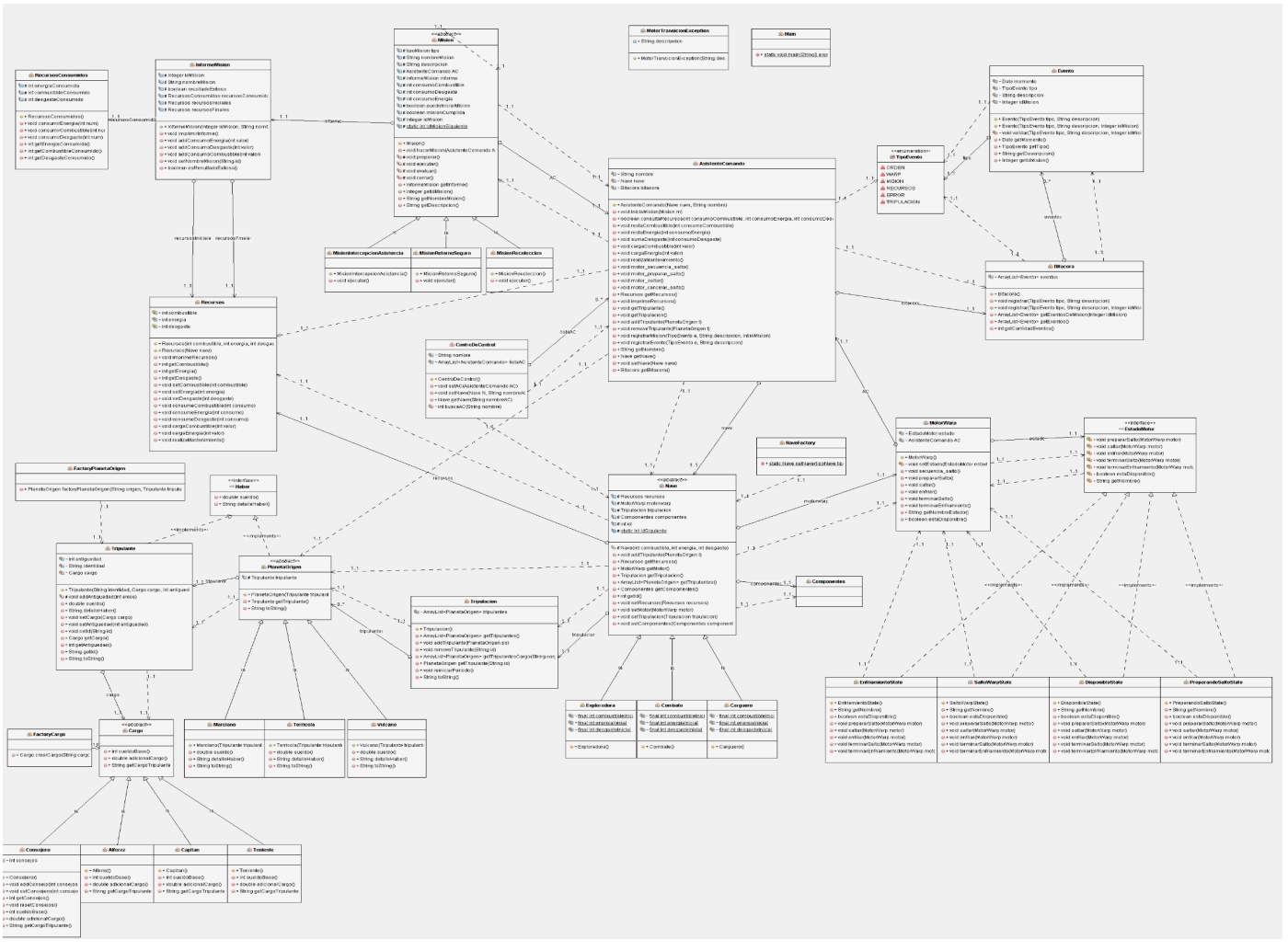
DESARROLLO

Síntesis

Durante gran parte del proyecto, la creación del software estuvo basado en decisiones que fueron tomadas luego de la lectura del enunciado y aclaraciones del cliente. Al comprender el problema, se implementaron las creaciones de las distintas clases del proyecto, tales como:

- Asistente de Comandos
- Nave (y sus 3 tipos, con su patrón Factory correspondiente)
- Tripulacion (con sus 3 tipos de Cargo y 3 tipos de Origen)
- Mision (y sus 3 diferentes misiones)
- Bitacora
- Evento
- MotorWarp (con sus distintos estados según su patrón State)

Dando como resultado un diagrama UML, que servirá como ‘mapa’ para el desarrollo de software:



Implementación de Patrones de diseño

Los siguientes patrones fueron utilizados para la realización del proyecto de acuerdo a los requisitos planteados dentro del Trabajo Práctico Grupal. Más detalles se presentan en el archivo adjunto "*decisiones-diseño.md*"

- *Patrón Decorator*

  Se implementó el patrón Decorator para asociar dinámicamente a cada tripulante un planeta de origen, modificando su comportamiento según el origen asignado. En particular, este patrón permite personalizar el sueldo del tripulante y obtener información detallada sobre sus características sin modificar directamente la clase base.

- *Patrón Factory*

  Se implementaron fábricas para la creación de cargos y planetas de origen. También funciona para la creación de los distintos tipos de nave (Exploradora, Carguero, y Combate) Estas fábricas centralizan la instancia de los objetos correspondientes y permiten incorporar nuevos tipos de forma dinámica, favoreciendo la extensibilidad y reduciendo el acoplamiento entre las clases.

- *Patrón State*

  Se optó por usar el patrón State para modelar el motor warp de la nave, evitando estructuras condicionales complejas (if / switch) para evaluar transiciones de estado. A través de la interfaz EstadoMotor y sus implementaciones concretas (DisponibleState, PreparandoSaltoState, SaltoWarpState y EnfriamientoState), cada fase encapsula su propio comportamiento, garantizando el flujo secuencial del ciclo de salto y protegiendo la integridad del sistema al disparar una MotorTransicionException ante cualquier acción no permitida.

- *Patrón Template-Method*

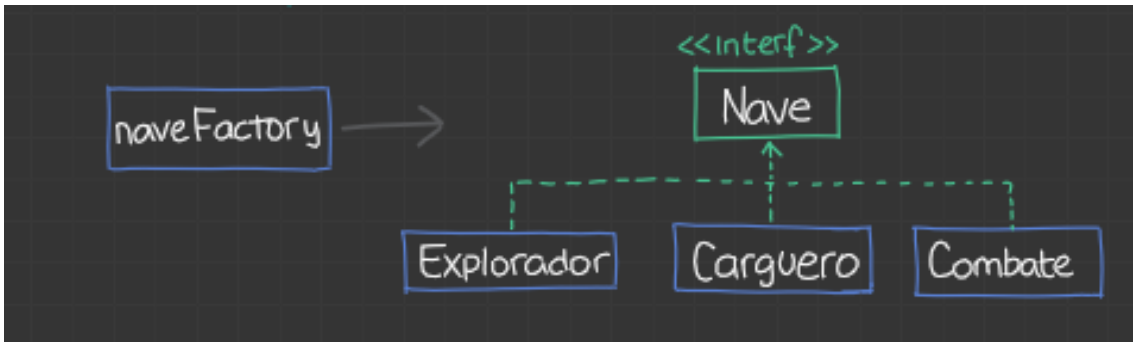
Este patrón es utilizado para realizar las misiones propuestas hacia un Asistente de Comandos en específico. Se utiliza este patrón para hacer el 'paso a paso' necesario para el cumplimiento de la misión, siendo estos:

- Preparar(): Se hacen las verificaciones necesarias de recursos de la nave que realiza la misión. La nave asignada tiene que tener los recursos suficientes para realizar el siguiente paso. En caso de no tenerlos, se cancela la misión.
- Ejecutar(): Se hace la ejecución de la misión. En este mismo, a la nave se restan los recursos que requiere la misión para ser completada.
- Evaluar(): Se hacen los cálculos necesarios para hacer comparaciones entre el estado inicial de la nave con el estado final. También se muestra un registro completo acerca de la misión ejecutada, su resultado y los recursos finales.
- Cerrar(): Cierra la misión y termina el algoritmo.

Organización y responsabilidades de las clases

- *Implementación de la clase Nave*

Uno de los elementos principales del proyecto es la Nave, protagonista que integra el Asistente de Comandos. Esta clase abstracta tiene como clases hijas a las naves Exploradora, Carguero y Combate. Todas estas naves son creadas utilizando el patrón **Factory,** justamente para facilitar la creación sin que el cliente conozca su procedimiento de instanciación.



- *Implementación de la clase Mision*

La clase misión es una clase abstracta que define una secuencia de acciones que se tienen que hacer en un orden específico. Para esto se usa el template method y nos garantiza que siempre se siga este orden, haciendo que las misiones que son tienen el mismo comportamiento se implementen en la clase abstracta mientras que las que sean distintas se implementen en la misión específica que herede de la clase abstracta misión.

Para contemplar el que una misión no pueda ser iniciada, o que al iniciarla no se pueda completar, sin que esto no haga que no se ejecuten las demás secuencias lo que hice fue que:

preparar(): asigna a una variable booleana puedeIniciarMision si es que puede empezar.

ejecutar(): si puedeIniciarMision == true entonces puede consumir los recursos para iniciar misión, sino la función avisa que no puede ejecutarse.

- *Implementación de la clase MotorWarp*

El desarrollo del subsistema consistió en la implementación de la clase MotorWarp y del ciclo de vida de los estados del motor de la nave. Para ello, se aplicó el patrón de diseño State, con el objetivo de modelar el comportamiento dinámico del motor y evitar el uso de múltiples estructuras condicionales para evaluar cada transición de estado.

La clase MotorWarp actúa como el contexto que delega las operaciones de transición a su estado actual mediante una referencia polimórfica a la interfaz EstadoMotor. Esta clase proporciona operaciones para gestionar el ciclo completo de propulsión (preparar salto, saltar, enfriar y finalizar enfriamiento), garantizando una secuencia estricta y segura. Ante cualquier intento de transición inválida, se dispara una excepción propia (MotorTransicionException), protegiendo la integridad operativa de la nave.

Interfaz EstadoMotor: define los métodos comunes de transición y consulta que deben implementar los distintos estados del motor (DisponibleState, PreparandoSaltoState, SaltoWarpState y EnfriamientoState). Cada implementación concreta encapsula la lógica correspondiente a su fase y define hacia qué estado es válido avanzar.

- *Implementación de la clase Tripulación*

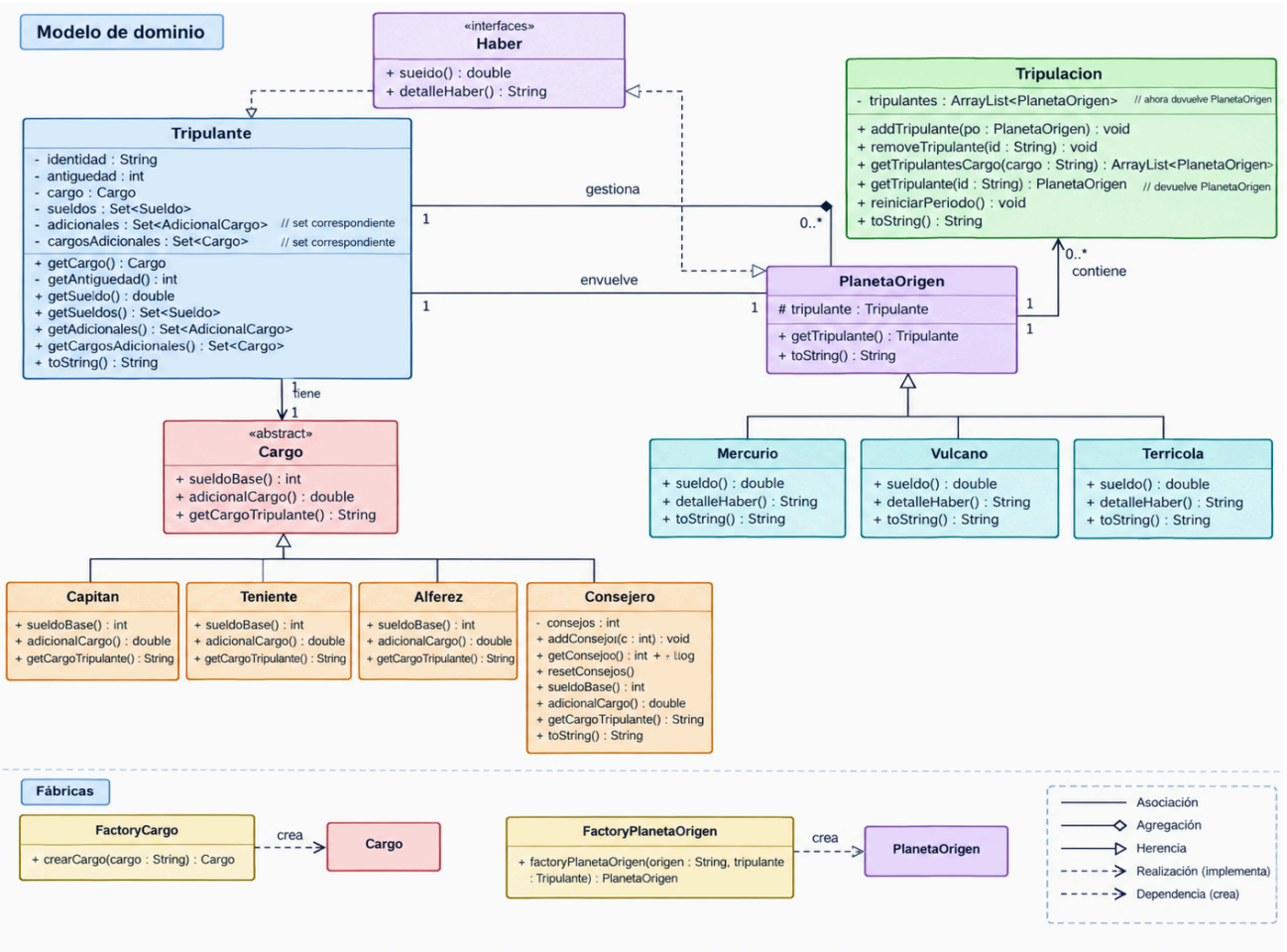
El desarrollo del sistema consistió en la implementación de la clase **Tripulación** y de los distintos tipos de tripulantes, incluyendo sus respectivos cargos y planetas de origen. Para ello, se aplicaron los patrones de diseño **Decorator** y **Factory**, con el objetivo de facilitar la creación de cargos y la asignación de orígenes de manera flexible y extensible.

La clase **Tripulación** administra a los integrantes de una nave mediante una lista (`ArrayList`) de objetos de tipo `PlanetaOrigen`, que encapsula a los tripulantes correspondientes.

Esta clase proporciona operaciones para agregar, eliminar, consultar y gestionar los integrantes de la tripulación, permitiendo administrar de manera organizada a los tripulantes de una nave interestelar.

- **Interfaz `Haber`:** define los métodos comunes que deben implementar tanto los objetos base como los decoradores. Incluye las operaciones relacionadas con la obtención del sueldo y la descripción detallada del tripulante.
- **Clase `Tripulante`:** representa el objeto base del patrón Decorator e implementa la interfaz `Haber`. Contiene los datos principales del tripulante, como su identidad, antigüedad y cargo. Además, mantiene una relación de agregación con la clase `Cargo`, ya que cada tripulante tiene un cargo asignado.
- **Clase abstracta `Cargo`:** Define el comportamiento y las características comunes de los cargos. Es la clase base de `Capitán`, `Teniente`, `Alférez` y `Consejero`, que implementan las particularidades de cada puesto, como el sueldo base y los adicionales correspondientes.
- **Clase `PlanetaOrigen`:** Actúa como decorador del patrón Decorator. Implementa la interfaz `Haber` y mantiene una referencia al tripulante que encapsula, permitiendo modificar dinámicamente su comportamiento en función del planeta de origen.
- **Clases `Marciano`, `Vulcano` y `Terrícola`:** Representan los distintos tipos de planetas de origen. Heredan de `PlanetaOrigen` y definen las particularidades correspondientes a cada origen, como los ajustes en el sueldo y la información detallada del tripulante.
- **Clase `Tripulación`:** Se encarga de administrar los tripulantes de una nave, ofreciendo métodos para agregarlos, eliminarlos, consultarlos y gestionar la lista de integrantes.

- **Fábricas `FactoryCargo` y `FactoryPlanetaOrigen`:** Centralizan la creación de objetos de tipo `Cargo` y `PlanetaOrigen`, respectivamente, facilitando la incorporación de nuevas variantes sin necesidad de modificar directamente las clases que utilizan estos objetos.



ASISTENCIA DE LA INTELIGENCIA ARTIFICIAL

Tripulación

La inteligencia artificial se utilizó como herramienta de apoyo durante el desarrollo del sistema. Su contribución consistió en ayudar a definir y organizar las relaciones entre las clases para implementar correctamente el patrón Decorator, así como en detectar y corregir diversos errores identificados durante las pruebas de ejecución.

También brindó asistencia en la implementación y corrección del método `detalleHaber()` de las clases involucradas, contribuyendo a mejorar la consistencia del diseño y el funcionamiento general del sistema.

Motor Warp y Bitácora

Se usó la inteligencia artificial para:
1. Revisar el correcto cumplimiento de la especificación al terminar de escribir el código.
2. Revisar el correcto funcionamiento del código luego de hacer que el mismo compile, también la ausencia de casos edges.
3. Obtener ventajas y desventajas entre diferentes enfoques para poder decidir cual es el que más conviene.